

Anteater Chess

Version 1.0
User Manual

EECS 22L: Software Engineering Project in C
University of California, Irvine
Spring 2026

Team 22: DeepAnteater

Yao Li
Benjamin Feng
Yide Li
Yurang Li
Yasith Diunugala
Max Zhang

Table of Contents

Table of Contents.....	2
Glossary.....	3
1. Anteater Chess Software Overview.....	6
1.1. Introduction.....	6
1.2. Goals.....	6
1.3. Usage scenarios.....	7
1.4. Main features.....	11
2. Installation.....	13
2.1. System requirements.....	13
2.2. Setup and configuration.....	13
2.3. Uninstalling.....	13
3. Chess Program Functions and Features.....	14
3.1. Main Menu.....	14
3.2. Game Mode Selection Menu.....	15
3.3. Game Setup Menu.....	16
3.4. Gameplay UI.....	20
3.5. Move Generation and Execution Procedure.....	25
3.6. Turn Management.....	32
3.7. End Game.....	35
3.8. Error Handling.....	38
3.9. Move Log.....	40
4. Back Matter.....	44
4.1. Troubleshooting.....	44
4.2. Error Messages.....	44
4.3. Copyright and contact information.....	47
4.4. Index.....	47

Glossary

The following terms are used throughout this manual to describe the rules, pieces, and gameplay of Anteater Chess.

Term	Definition
Ant	Another name for a pawn in Anteater Chess.
Ant Eating	A special move unique to the anteater piece, allowing it to capture a chain of adjacent pawns (ants) along the same file or rank in a single move
Anteater	A new chess piece introduced in Anteater Chess. It moves one square in any direction (like a king) but can only capture pawns. It can capture multiple pawns in a single move through the ant eating mechanic
Bishop	A piece that moves diagonally across the board any number of squares.
Castling	A special move involving the king and a rook. The king moves two squares toward the rook, and the rook jumps to the square the king crossed.
Check	A situation in which a player's king is under direct attack by an opposing piece. The player must resolve the check on their next move.
Checkmate	A check from which the king cannot

	escape. The game ends, and the player whose king is in checkmate loses.
En Passant	A special pawn capture that occurs when a pawn advances two squares from its starting position, and an adjacent enemy pawn could have captured it had it moved only one square. The capture must occur on the immediately following turn.
File	A vertical column on the chessboard, labeled A through J in Anteater Chess.
King	The most important piece. It moves one square in any direction. If the king is in checkmate, the game is lost.
Knight	A piece that moves in an L-shape: two squares in one direction and one square perpendicular to that. It is the only piece that can jump over other pieces.
Pawn	Also called an ant. Moves forward one square (or two from its starting position) and captures diagonally. If it reaches the opposite end of the board, it is promoted to another piece.
Promotion	When a pawn reaches the opposite end of the board, it can be replaced by any piece other than the king, typically a queen.
Queen	The most powerful piece. It can move horizontally, vertically, and diagonally across the board any number of squares.
Rank	A horizontal row on the chessboard,

	numbered 1 through 8.
Rook	A piece that moves horizontally and vertically across the board any number of squares.
Stalemate	A situation where the player whose turn it is has no legal moves, and their king is not in check. The game is a draw.

1. Anteater Chess Software Overview

1.1. Introduction

Anteater Chess is an interactive board game that runs on Linux and is implemented in C. It allows a human player to play a modified version of chess against another human player or a computer opponent, and also supports matches between two computer players.

Compared to traditional chess, Anteater Chess features an extended board with ten files (A–J) and introduces a special piece called the anteater, which can capture multiple pawns in a single move. These additions create a more dynamic gameplay experience.

The software provides an interactive environment where users can view the board, make moves, and receive almost real-time feedback while all game rules are enforced.

Anteater Chess is based on the standard rules of chess as defined by FIDE (Fédération Internationale des Échecs). For the official chess rules, please refer to:

<https://www.fide.com/FIDE/handbook/LawsOfChess.pdf>. Unless otherwise specified in this manual, all standard chess rules apply. The modifications unique to Anteater Chess are described in the sections below.

1.2. Goals

The goal of the Anteater Chess software is to provide users with an innovative and interactive Anteater Chess experience. The program is designed to support a graphical user interface (GUI) through which users can interact with the game in two ways: by clicking with the mouse and by entering commands.

The software should allow a player to play either against a computer opponent or against another local player. After each move, the system provides immediate feedback by displaying the new move made by the computer player or the second local player, and the graphical chess board is updated in real time.

Overall, the program aims to offer a clear, responsive, and user-friendly gameplay experience that allows players to easily follow the game state and interact with the system efficiently.

1.3. Usage scenarios

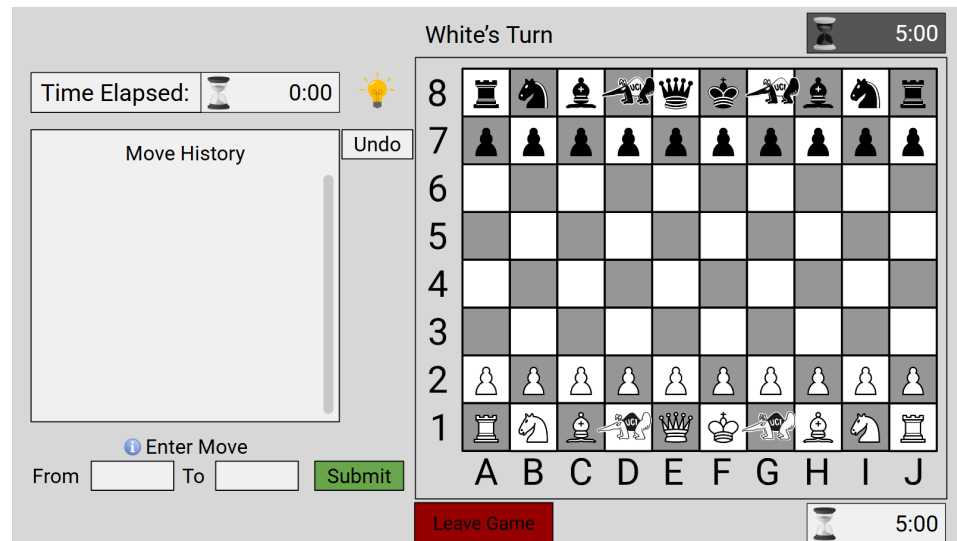
The following scenarios describe typical interactions between a user and the Anteater Chess program.

1.3.1. Program Start

- The user launches the program by using the command
\$./bin/chess
- on a Linux server via the command line.
- A graphical user interface (GUI) menu is displayed upon startup.

1.3.2. Game Setup

- The user selects a game mode by clicking on an option in the menu (e.g., human vs. computer or human vs. human).
- In human-versus-computer mode, the user selects a side (black or white).
- The game initializes and displays the chessboard.

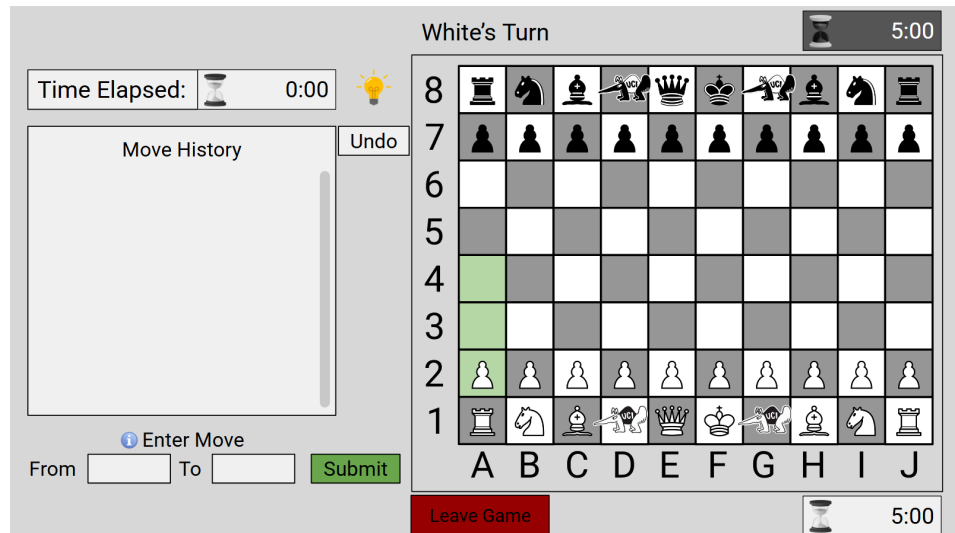


1.3.3. Gameplay Interaction

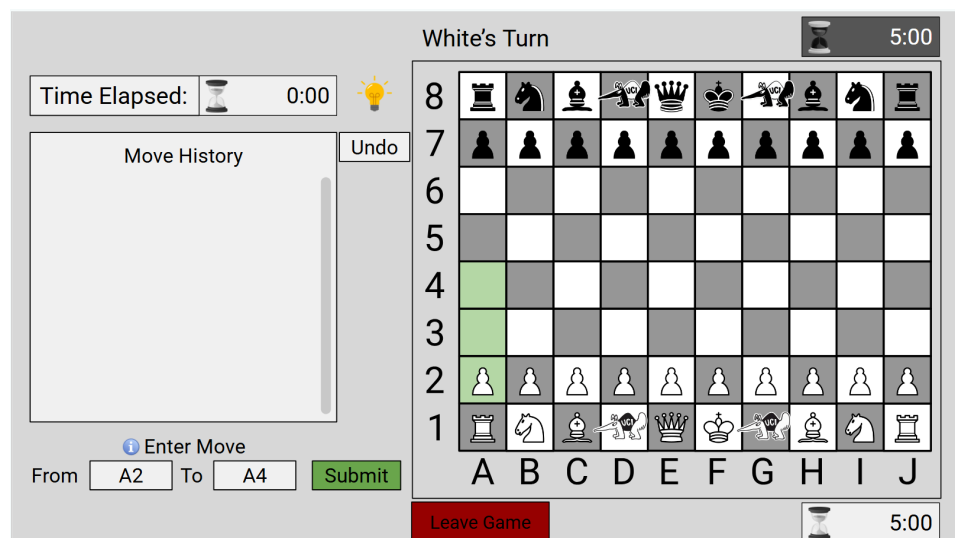
The program displays a graphical chess board with 10 files (A–J) and 8 ranks (1–8), including coordinate labels and a turn indicator (e.g., “White’s turn”).

The user makes a move using one of the following methods:

- Mouse interaction:
 - Left-click to select a piece
 - Right-click to select a destination square
 - Valid target positions are highlighted in green



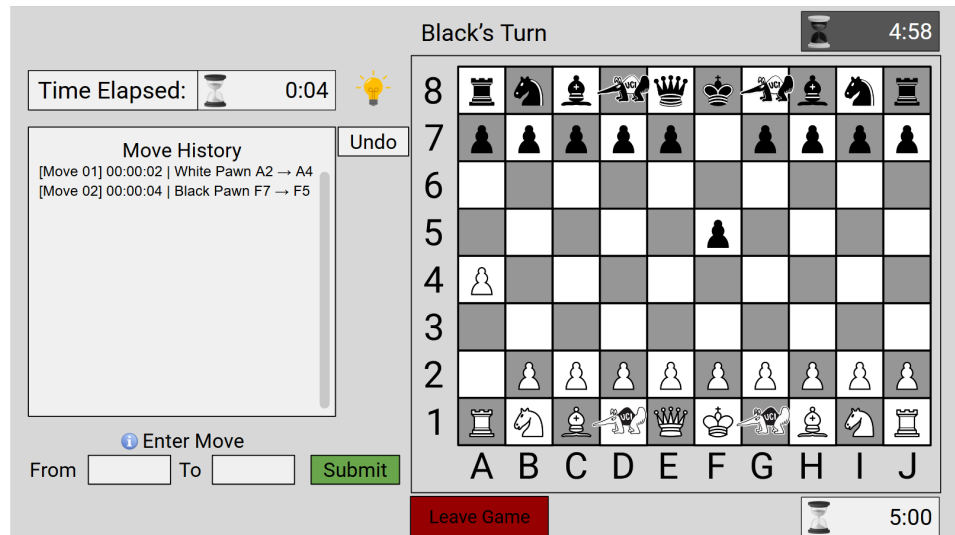
- Command input:
 - Enter a move in the format “From To” (e.g., “A2 A4”)



After a move is entered, the system validates it:

- If the move is invalid, an error message (“Invalid move”) is displayed, and the user must retry.
- If the move is valid, the piece is moved to the destination square.
- If an opposing piece is located on the destination square, it is captured and replaced. A message is displayed indicating the captured piece.

The updated board is displayed after each valid move.



1.3.4. Opponent Interaction

- In human-versus-computer mode:
 - The computer will generate and perform its move after the user completes a valid move in 1 minute.
 - The updated board is displayed along with a message indicating the computer's move.
- In human-versus-human mode:
 - Players alternate turns.
 - The system displays whose turn it is (black or white).

1.3.5. Game Termination

- The game continues until a termination condition is reached:
 - Checkmate
 - Draw
 - Stalemate: A player's turn where there are no legal moves, but their king is not in check
 - Insufficient material: Neither player has enough pieces to checkmate
 - King vs. King
 - King + Bishop vs King
 - King + Knight vs King
 - King + Bishop vs King + Bishop (Bishops on same color tile)
- When the game ends, the system displays the result:
 - "White wins"
 - "Black wins"
 - "Draw"
- After the game ends, the user is given options to:
 - Return to the main menu

- Start a new game



1.4. Main features

The features of the Anteater Chess software are categorized into three groups: required core features, advanced features, and user interface enhancements.

1.4.1. Core Features

The following features represent the fundamental functionality of the system and are required for a complete implementation:

- Full implementation of standard chess rules with Anteater Chess extensions
- Interactive graphical user interface (GUI) for board visualization and user input
- Support for human vs. computer gameplay
- Support for human vs. human gameplay
- Player side selection (white or black) before the game starts
- Real-time move validation and rule enforcement
- Turn-based game control with clear player indication
- Automatic move generation for the computer player
- Bounded response time for computer moves (less than one minute per move)
- Detection of game termination conditions (checkmate and draw)
- Logging of all moves in a human-readable format

1.4.2. Advanced Features (Enhancements)

The following features extend the functionality of the system and improve gameplay experience:

- Support for computer vs. computer gameplay
- Undo functionality to withdraw previous moves
- Adjustable difficulty levels for the computer player (e.g., beginner, intermediate, expert)
- Hint system for suggesting possible moves to the player
- Turn timers for both players
- Move history panel for tracking previously executed moves

1.4.3. User Interface Enhancements

The following features improve usability and provide visual feedback to the user:

- Legal move highlighting for selected pieces
- Visual turn indicator (e.g., "White's turn")
- Invalid move feedback messages
- Capture indicator for displaying captured pieces
- Global clock to display the elapsed game time
- Restart game option from the interface

1.4.4. Input Handling and Usability Enhancements

- Case-insensitive command input:
 - The system accepts move commands regardless of letter case (e.g., "a2 a3", "A2 A3", or "a2 A3" are all treated equivalently).
- Flexible whitespace handling:
 - The system tolerates variations in spacing, including leading, trailing, or multiple spaces before or after inputs (e.g., " A2 ").
- Input format guidance:
 - The interface provides prompts or examples indicating the correct command format (e.g., "Enter move: From "A2" to "A3"").
- Input validation feedback:
 - If the input format is incorrect or cannot be parsed, the system displays an appropriate error message and waits for the next user input.

2. Installation

2.1. System requirements

The following are the minimum system requirements for running Anteater Chess.

Component	Requirement
Operating System	Linux (Rocky Linux 8.10)
Processor Architecture	x86-64
Memory	512 MB RAM Minimum
Disk Space	100 MB recommended
Compiler	GCC 8.5.0 or later
Build Tool	GNU Make 4.2.1 or later
GUI Libraries	libgtk-3-0
Display	X11-capable display or terminal emulator

2.2. Setup and configuration

Follow these steps to install and configure Anteater Chess:

Step 1: Obtain the software package.

Download or copy the release archive Chess_V1.0.tar.gz to your system.

Step 2: Extract the archive.

```
$ gtar xvzf Chess_V1.0.tar.gz
```

Step 3: Enter the package directory.

```
$ cd Chess_V1.0
```

Step 3: Run the program.

```
$ ./bin/chess
```

2.3. Uninstalling

To uninstall Anteater Chess, remove the extracted package directory:

```
$ rm -rf Chess_Alpha/
```

3. Chess Program Functions and Features

This chapter provides a detailed description of each function and feature available in the Anteater Chess program.

3.1. Main Menu

Description:

When the program is launched, the main menu is displayed. The main menu contains two options: **New Game** and **Quit Game**.

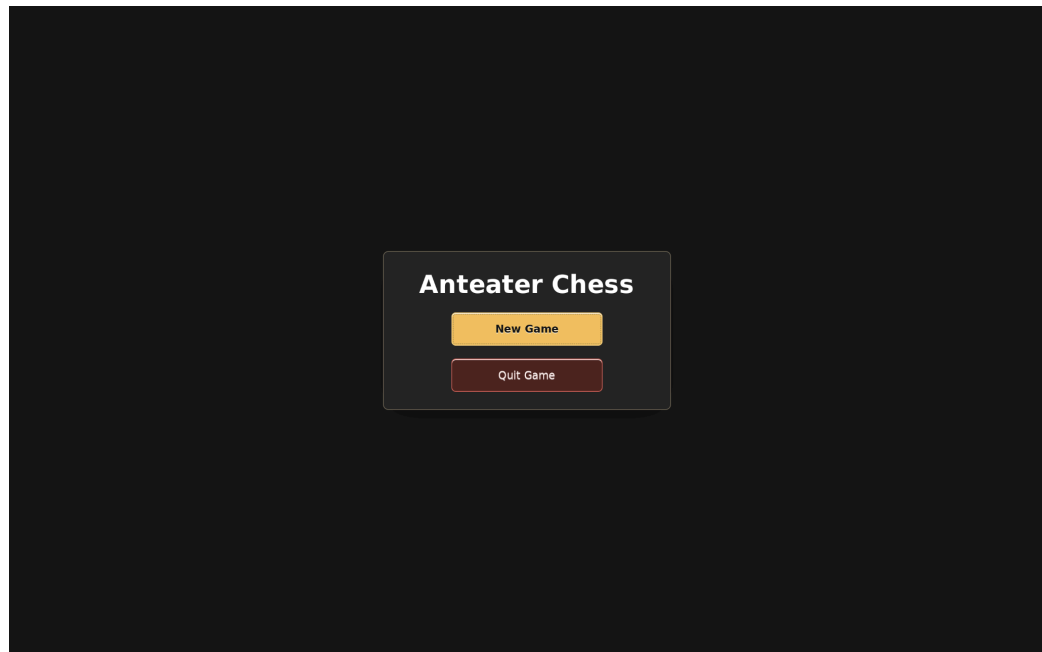
Case 1: New Game

User Input:

- Select “**New Game**” from the **Main Menu**

Program Output:

- The program displays a [Game Mode Selection Menu](#)



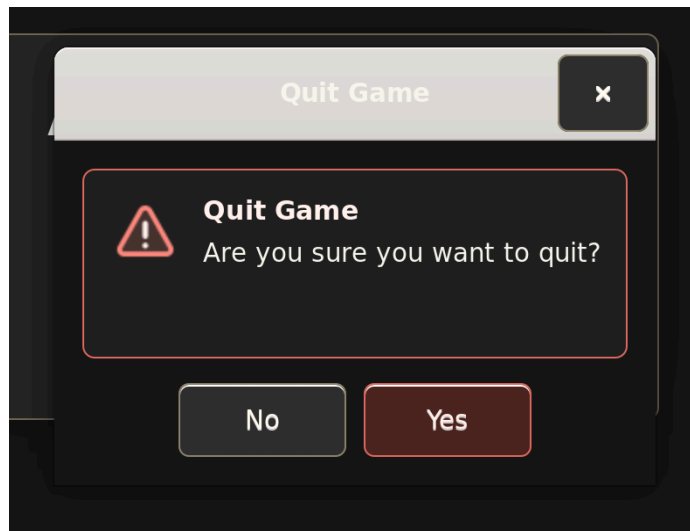
Case 2: Quit the Program

User Input:

- Select “Quit Game” from the main menu

Program Output:

- The program displays a confirmation message
- The program terminates and returns control to the operating system



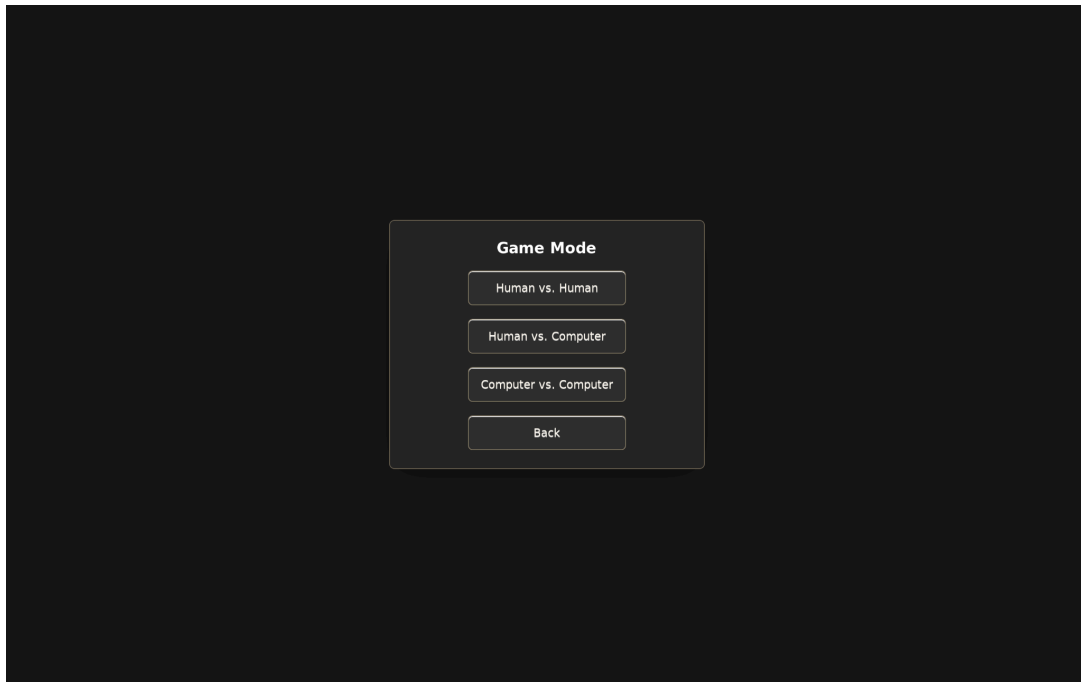
3.2. Game Mode Selection Menu

Description:

After selecting **New Game** from the **Main Menu**, the program displays the **Game Mode Selection Menu**. In this menu, the user selects the desired gameplay mode.

The available options are:

- Human vs. Computer
- Human vs. Human
- Computer vs. Computer
- Back (return to the main menu)

**User Input:**

The user selects one of the following options:

- Click “Human vs. Computer” to play against the computer
- Click “Human vs. Human” for a two-player game
- Click “Computer vs. Computer” to observe an automated match
- Click “Back” to return to the main menu

Program Output:

Depending on the selected game mode, the program proceeds as follows:

- Human vs. Computer/ Human vs. Human/ Computer vs. Computer
 - The program transitions to the [Game Setup Menu](#)
 - Back:
 - The program returns to the [Main Menu](#)
 - No game is initialized
-

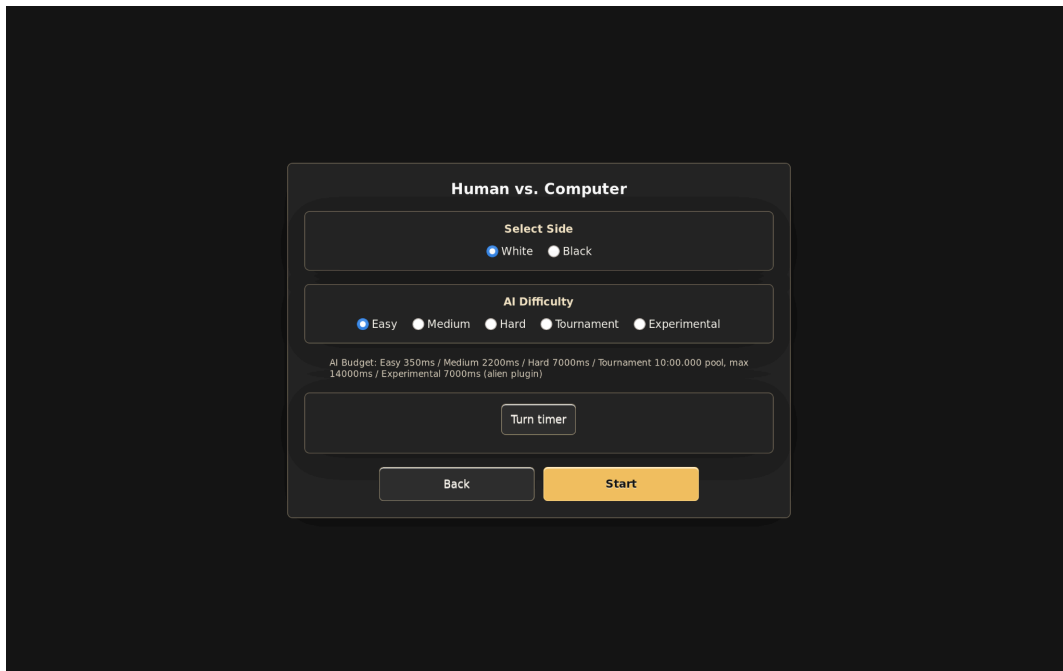
3.3. Game Setup Menu

Description:

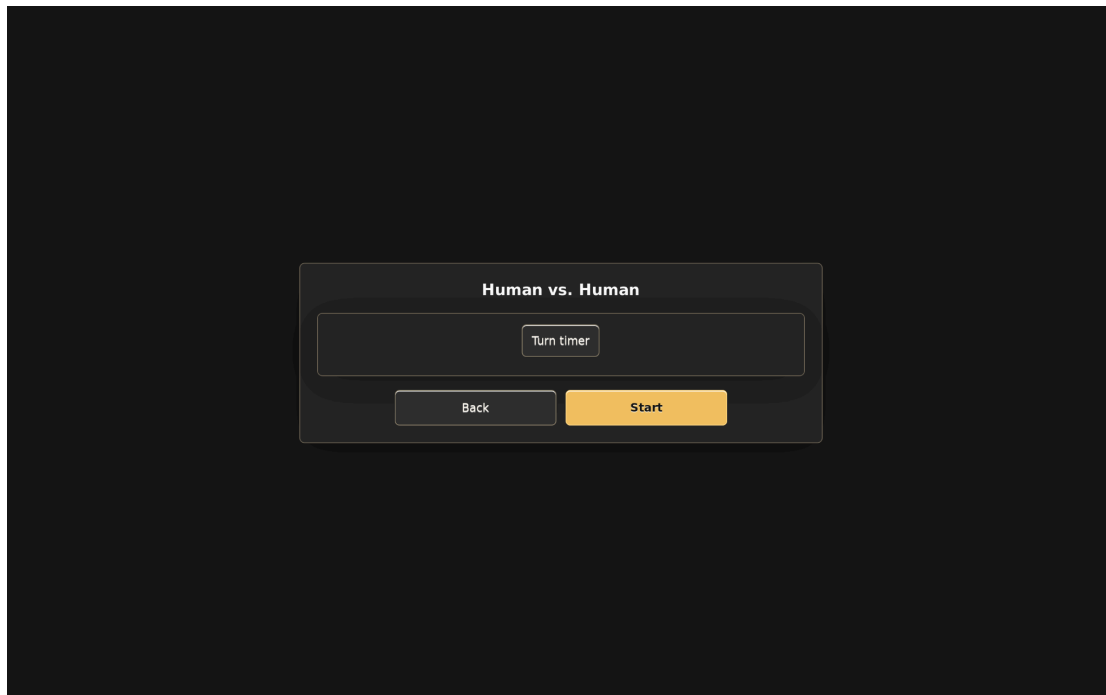
After selecting a game mode that requires additional configuration (i.e., Human vs. Computer or Computer vs. Computer), the program displays the **Game Setup Menu**. In this menu, the user configures game settings before the game begins.

The available options depend on the selected game mode:

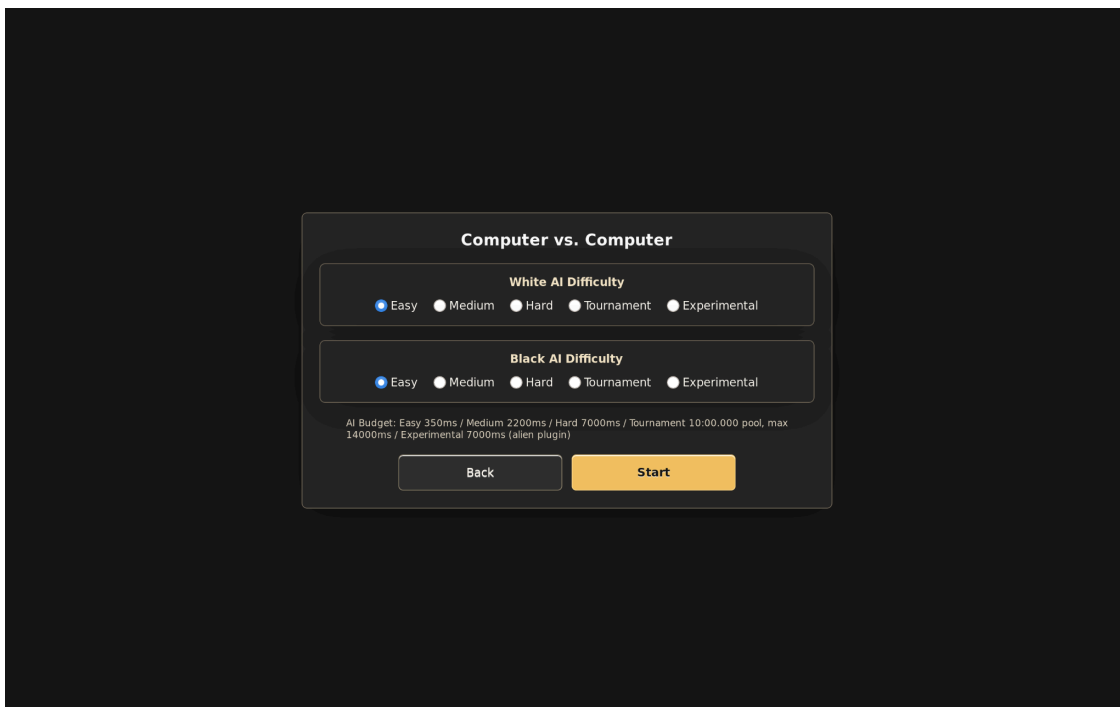
- Human vs. Computer:
 - Select player side (white or black)
 - Select AI difficulty level
 - Enable or disable the **TurnTimer**
 - Back option to return to the **Game Setup Menu**
 - Start Game option to start the game



- Human vs. Human:
 - Enable or disable the **Turn Timer**
 - Back option to return to the **Game Setup Menu**
 - Start Game option to start the game



- Computer vs. Computer:
 - Select AI difficulty level for the white player
 - Select AI difficulty level for the black player
 - Back option to return to the **Game Setup Menu**
 - Start Game option to start the game



- Back option to return to the **Game Mode Selection Menu**

User Input:

The user configures the game by interacting with the available options:

- Select player side (if applicable)
- Select AI difficulty level (if applicable)
- Toggle timer on/off and select time (optional)
- Click Start Game to confirm the configuration
- Click Back to return to the **Game Mode Selection Menu**

Program Output:

Depending on the user's actions, the program proceeds as follows:

Start Game:

- The program initializes the standard [Gameplay UI](#) without a [Turn Timer](#) displayed.
- If the timer is enabled:
 - The **Turn Timer** is displayed in the **Gameplay UI**
 - The countdown starts for the current player

Back:

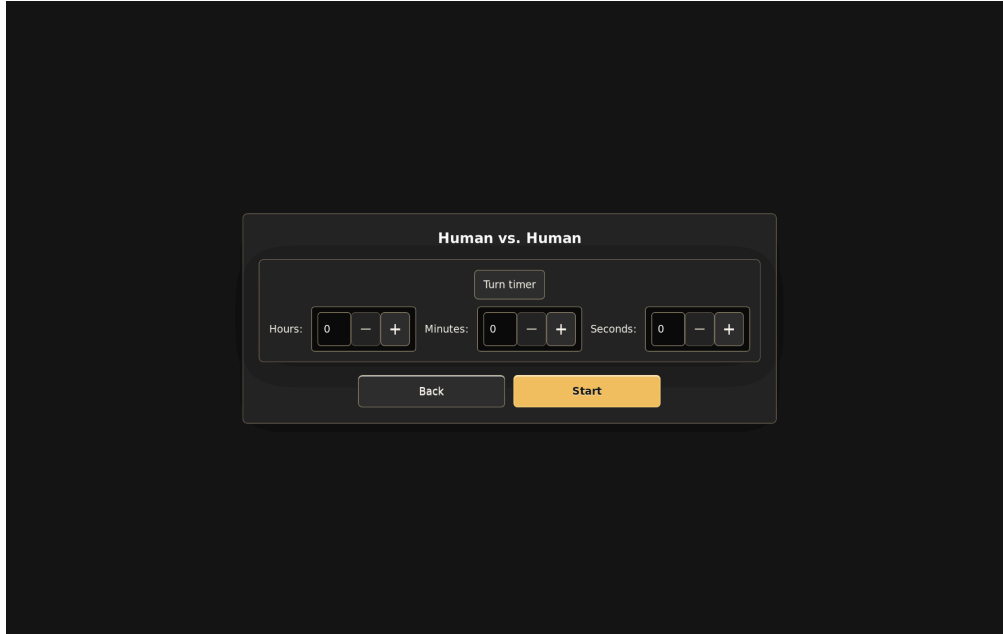
- The program returns to the [Game Mode Selection Menu](#)
- No game is initialized

Turn Timer (Toggle + Dropdown)**Description:**

The **Game Setup Menu** of Human vs. Computer mode and Human vs. Human mode includes a **Turn Timer Toggle** that allows the user to enable or disable time constraints for each turn.

User Input:

- Click the **Turn Timer Toggle** to enable or disable the timer
- If the timer is enabled:
 - A **dropdown menu** expands below the toggle
 - The user enters the time limit per turn using three input fields:
 - [] h (hours)
 - [] min (minutes)
 - [] s (seconds)



Program Output:

- If the timer is disabled:
 - No time limit is applied during gameplay
- If the timer is enabled:
 - The selected time limit is stored in the game configuration
 - The [Turn Timer](#) component is activated in the [Gameplay UI](#)
 - Each player's turn is constrained by the selected time limit

3.4. Gameplay UI

After the game is initialized, the program displays the main gameplay interface, referred to as the **Gameplay UI**. This interface presents the current game state and provides all controls required for user interaction during gameplay.

All of the following components are displayed in the **Gameplay UI** throughout the game. They are grouped below by function type for clarity.



3.4.1. Main Functions

Anteater Chessboard

Description:

Located at the right side of the **Gameplay UI**, the **Anteater Chessboard** displays the current game state by showing all pieces on a 10×8 grid (files A–J and ranks 1–8).

User Input:

- The user interacts with the board indirectly through [Move Inputs](#)

Program Output:

- Displays the current position of all pieces
- Updates dynamically after each valid move, including piece movement and captures

Turn Indicator

Description:

Located above the **Anteater Chessboard**, the Turn Indicator shows which player's turn it is during the game.

User Input:

- No direct user input is required

Program Output:

- Follows the behavior of [Turn Indicator Behavior](#)
-

Move Input Widget

Description:

Located at the bottom-left side of the **Gameplay UI**, the **Move Input Widget** allows the user to enter and submit a move command.

User Input:

- The user enters a starting position in the **From field**
- The user enters a destination position in the **To field**
- The user **submits** the move

Program Output:

- Any output passed through the [Move Generation and Execution Procedure](#)
-

Leave Game Button

Description:

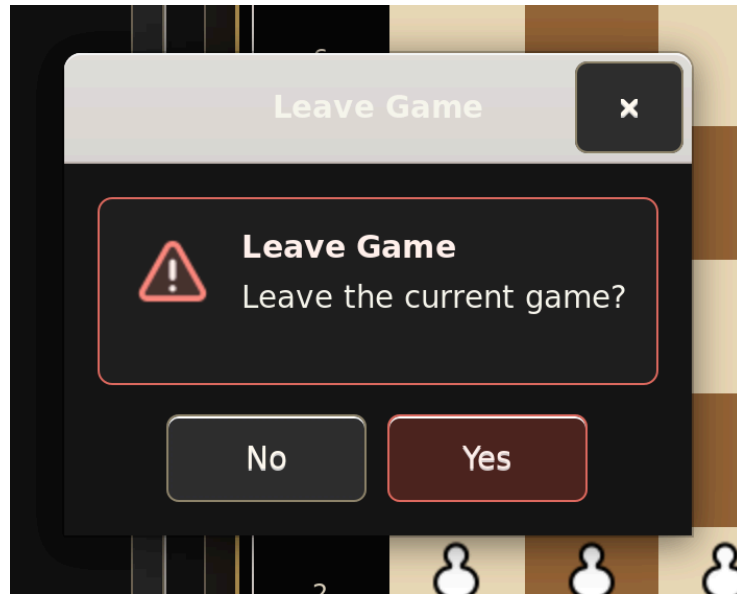
Located at the bottom of the **Gameplay UI**, the **Leave Game Button** allows the user to exit the current game session.

User Input:

- The user clicks the **Leave Game button**

Program Output:

- A confirmation dialog is displayed
- If the user cancels, the dialog closes and gameplay resumes
- If the user confirms, the current session enters the [Game Termination State](#)



3.4.2. Advanced Functions

Clock

Description:

Located to the left of the [Turn Indicator](#), the clock displays the total time elapsed since the beginning of the current game.

User Input:

- No direct user input is required

Program Output:

- The total time elapsed since the beginning of the game is displayed during the game
-

Turn Timer

Description:

A separate **Turn Timer** is displayed for each side, with one timer located above the **Anteater Chessboard** and the other located below it. Each timer tracks the remaining time for its corresponding player, and only the timer of the current player counts down during that turn.

User Input:

- No direct user input is required

Program Output:

- Follows the [Turn Timer Behavior](#)
-

Undo Button**Description:**

Located on the left side of the [Anteater Chessboard](#), the **Undo Button** allows the user to revert previous moves.

User Input:

- The user clicks the **Undo Button**

Program Output:

- Follows the [Undo Behavior](#)
-

Move History Panel**Description:**

Located on the left side of the [Anteater Chessboard](#), the **Move History Panel** displays a history of moves made during the game.

User Input:

- No direct user input is required

Program Output:

- Displays a panel showing a chronological list of previous moves
 - Updates the list as new moves are made
-

Hint (Bulb Icon)**Description:**

Located on the left side of the [Anteater Chessboard](#), the **Hint** function is accessed via a bulb icon and provides suggested moves for the current player.

User Input:

- The user clicks the bulb icon

Program Output:

- Displays the most recommended move for the current player
-

3.4.3. User-Friendly Features

Overview:

Move Format Information

Description:

The **Move Format Information** function provides guidance on the correct format for move command input. The function is accessed via an information icon located next to the [Move Input Widget](#) at the bottom-left of the **Gameplay UI**.

User Input:

- The user hovers over the information icon

Program Output:

- Displays a tooltip showing the correct move format (e.g., "A2 A3")
-

3.5. Move Generation and Execution Procedure

This section describes how a move is generated and processed by the system. A move may originate from either a human player or a computer player. Although the two types of players generate moves differently, both follow the same core processing logic for legality checking, execution, and game-state update. The overall procedure therefore consists of player-specific input or move-generation steps, followed by shared validation and execution steps.

3.5.1. Overview of Shared and Player-Specific Steps

The move procedure contains both distinct and shared stages.

Human-player-specific steps

- Piece selection
- Destination selection or command entry
- Legal move highlighting

Computer-player-specific step

- Automatic move generation based on the current game state

Shared steps

- Legal move calculation
- Move validation

- Move execution
- Board update
- Move log update
- Transition to Game Flow and Turn Management

In other words, the human player and the computer player differ mainly in how a candidate move is obtained. Once a move is constructed, both types of players use the same validation and execution logic.

3.5.2. Human Player Move Flow

The human player provides move information through direct interaction with the [Gameplay UI](#). The process begins with user input and proceeds through a sequence of validation and execution steps.

The system supports two human input methods:

Mouse Interaction:

Description:

The user interacts directly with the [Anteater Chessboard](#)

User Input:

- Left-click: Select a piece
- Right-click: Specify the destination square

Program Output:

- The system interprets the first click as the selection input
- The system interprets the second click as the move input

Command Input:

Description:

The user inputs a move using the [Move Input Widget](#).

User Input:

- Enter the starting position in the **From field**
- Enter the destination position in the **To field**
- Click the **Submit button**

Program Output:

- The system interprets the **From field** as the selection input
- The system interprets the **To field** and **Submit** action as the move input



A human player's move typically follows the sequence below:

1. The user selects a piece
2. The system validates the selection
3. The system computes legal moves for the selected piece
4. The system highlights legal destinations
5. The user chooses a destination square
6. The system constructs the requested move
7. The system validates the move
8. If valid, the move is executed and the UI is updated
9. Control passes to [Turn Management](#)

3.5.3. Computer Player Move Flow

The computer player does not interact with the user interface in the same way as a human player. Instead, it generates a move internally from the current board state.

Description:

When it is the computer player's turn, the system automatically determines a move for that side.

User Input:

- No direct user input is required

Program Output:

- The system computes the set of legal moves available to the computer player
- The system selects one move according to the computer player's decision logic

- The selected move is passed to the same [Move Validation](#) function used for human moves
- If valid, the move is executed through the same [Move Execution and UI Update Procedure](#)

A computer player's move typically follows the sequence below:

1. The system identifies that the current turn is controlled by the computer
2. The system computes legal moves for that side
3. The system selects one move internally
4. The system constructs the selected move
5. The system validates the move
6. If valid, the move is executed and the UI is updated
7. Control passes to [Turn Management](#)

Key difference from the human flow

The computer player does not perform:

- piece selection through clicks or command input
- legal move highlighting
- destination input through the Move Input Widget

These UI-specific steps apply only to human players.

3.5.4. Select Validation

Description:

This function validates whether the selected square corresponds to a valid piece that can be moved by the current player.

User Input:

A selected square from:

- Left-click (mouse), or
- From field (command input)

Program Output:

- **Case 1: Empty square**
 - No piece is found
 - The system performs no action and waits for the next selection
- **Case 2: Opponent's piece**
 - The selected piece does not belong to the current player

- The system performs no action and waits for the next selection
 - **Case 3: Valid selection**
 - The selected piece belongs to the current player
 - The selected cell information (position, piece type, color) is passed to the **Legal Move Calculation** function
-

3.5.5. Legal Move Calculation

For a human player

User Input:

- A valid piece selection from the **Select Validation** function

Program Output:

- A list of legal destination squares for the selected piece
 - The result is passed to the **Legal Move Highlight** function
-

For a computer player

User Input:

- The current board state and the identity of the current player

Program Output:

- A list of legal moves available to the computer player
 - The result is used by the computer player's decision logic to select one move
 - The selected move is passed to the **Move Validation** function
-

3.5.6. Legal Move Highlight

Description:

This function visually displays all legal moves for a selected human-player piece on the **Anteater Chessboard**.

User Input:

- A list of legal moves from the **Legal Move Calculation** function

Program Output:

- All legal destination squares are highlighted in green
- The **Anteater Chessboard** is updated to display highlights
- The system waits for a move input (destination selection)



3.5.7. Move Construction

Description:

This function processes the destination information associated with a move and constructs the requested move in the form From → To.

For a human player

User Input:

- Destination square from:
 - Right-click (mouse), or
 - To field + Submit (command input)

Program Output:

- The selected move (From → To) is constructed
- The move is passed to the **Move Validation** function

For a computer player

User Input:

- A move selected internally by the computer player from the legal move list

Program Output:

- The selected move (From → To) is retrieved
- The move is passed to the **Move Validation** function

3.5.8. Move Validation

Description:

This function verifies whether the requested move is valid by comparing it against the legal moves permitted under the current game state.

User Input:

- A move (From → To)
- The corresponding legal move information from the Legal Move Calculation function

Program Output:

- **Case 1: Invalid move**
 - The move is not included in the legal move list
 - An error message, “Invalid move”, is displayed in the [Error Message Popup](#)
 - The system waits for a new move input or a new computer-generated selection
 - **Case 2: Valid move**
 - The move is passed to the **Move Execution and UI Update Procedure**
 - The move is recorded in the [Move Log](#)
 - A [Game Outcome Detection](#) is triggered
-

3.5.9. Move Execution and UI Update Procedure

Description:

This function applies the validated move to the game state and updates the [Gameplay UI](#) accordingly.

User Input:

- A validated move

Program Output:

- The piece is moved from the source square to the destination square
- Any captured piece is removed
- Special rules are applied when necessary:
 - Pawn promotion
 - Castling
 - Ant eating
- The **Gameplay UI** is updated:
 - The **Anteater Chessboard** reflects the new position
 - Highlights are cleared

- The program proceeds to the procedure of **Turn Management**
-

3.6. Turn Management

3.6.1. Turn Alternation

Description:

Similar to classic chess, when a game begins, White (whether controlled by a human or a computer) always takes the first turn.

During a turn, the current side is allowed to make exactly one move. After a valid move is executed, the turn is passed to the opposing side. This cycle continues until one of the endgame conditions described in Section 3.7 is met.

Turn alternation is the core mechanism that drives all turn-based updates in the system.

User Input:

- No direct input is required from the user

Program Output:

A Turn Alternation is triggered under the following conditions:

- A valid move is executed by either a human player or a computer player
- The turn timer reaches 0 (forced turn termination)
- An undo operation restores a previous turn state

When a turn alternation occurs:

- The current player switches to the opposing side
 - Turn-dependent systems (Turn Indicator and Turn Timer) are updated accordingly
-

3.6.2. Turn Indicator Behavior

Description:

To indicate which side is currently playing, a [Turn Indicator](#) is displayed. The indicator shows information such as “White’s turn” or “Black’s turn”.

The **Turn Indicator** is always synchronized with the current turn and is updated based on turn alternation events.

User Input:

- No direct input is required from the user

Program Output:

The **Turn Indicator** behaves as follows:

- **At game start:**
 - The **Turn Indicator** is initialized to display “White’s turn”, since White always moves first.
- **After a valid move is executed:**
 - The **Turn Indicator** is updated to display the opposing side’s turn.
- **After the turn timer reaches 0:**
 - The current turn is terminated, a **Turn Alternation** occurs, and the **Turn Indicator** is updated to display the opposing side’s turn.
- **After an undo operation:**
 - The **Turn Indicator** is updated to reflect the player whose turn has been restored.
- **If no Turn Alternation occurs (for example, after an invalid move, piece selection, or move highlighting):**
 - The **Turn Indicator** remains unchanged.

3.6.3. Turn Timer Behavior

Description:

When the **Turn Timer** is [enabled](#), a countdown timer is [displayed](#). The timer indicates the maximum duration allowed for a single turn.

The duration of the timer is defined by the settings in the [Game Setup Menu](#). Each turn starts with the same time limit, regardless of how much time was used in previous turns.

The Turn Timer is directly controlled by turn alternation events.

User Input:

- The user is expected to complete a move before the timer reaches 0.

Program Output:

During a turn:

- The timer for the current player counts down continuously

When a turn alternation occurs:

- The Turn Timer resets to the full duration and starts to count down

When the timer reaches 0:

- The current turn is immediately terminated
- An error message is displayed through the [Error Message Popup](#)
- A [Turn Alternation](#) is triggered
- Control is passed to the opposing side

If no **Turn Alternation** occurs (e.g., invalid move or selection), the timer continues running without reset.

3.6.4. Undo Behavior

Description:

The Undo function allows the system to revert the game state to a previous turn. This operation reverses the most recent move and restores the previous board configuration.

Undo is treated as a **reverse** [Turn Alternation](#).

User Input:

- The user clicks on the [Undo Button](#)

Program Output:

When an Undo operation is performed:

- The most recent move is removed
- The board state is restored
- The current player is reverted to the player who made the undone move
- A [Turn Alternation](#) is triggered **reversely**

As a result:

- The [Turn Indicator](#) is updated to reflect the restored player
- The [Turn Timer](#) switches back to that player and restarts

Notice:

- The [Clock](#) is **Never** affected by the **Undo** operation

3.7. End Game

This section describes the conditions under which the game ends, how the program detects and responds to these conditions, and the options presented to the user afterward.

3.7.1. Game Outcome Detection

Description:

Game Outcome Detection evaluates whether the game has reached a terminal condition based on the current board state.

This detection is triggered immediately after each [valid move](#) execution (by either a human or computer player).

User Input:

- No direct user input is required.

Program Output:

The program evaluates the updated game state:

- If no terminal condition is detected:
 - The game continues normally
 - If a terminal condition is detected (e.g., checkmate or draw):
 - The system enters the Game Termination State
-

3.7.2. Manual Termination Detection

Description:

Manual Termination Detection handles user-initiated game termination.

This process is independent of the game state and can be triggered at any time during gameplay.

User Input:

- The user clicks the [Leave Game Button](#).

Program Output:

When the Leave Game button is pressed:

- A confirmation dialog is displayed
 - If the user cancels:
 - The dialog is closed
 - The game resumes
- If the user confirms:
 - The system enters the **Game Termination State**

3.7.3. Game Termination State

Description:

The **Game Termination State** is a unified system state entered when the game is determined to have ended, either through **Game Outcome Detection** or **Manual Termination Detection**.

All game-ending events transition into this state before any user interface is displayed.

User Input:

- No further gameplay input is accepted in this state.

Program Output:

- All gameplay interactions are disabled
- Move input is no longer accepted
- Computer move generation stops
- Turn alternation stops
- Turn Indicator and Turn Timer stop updating
- The final board state is preserved

After these updates, the **End Game Menu** is displayed.

3.7.4. End Game Menu

Description:

The **End Game Menu** is displayed after the game enters the **Game Termination State**.

The menu is implemented as a modal overlay that appears above the [Gameplay UI](#).

It does not replace the existing game screen; instead, it is layered on top of it.

The **End Game Menu** is smaller in size than the **Gameplay UI** and is centered on the screen.

The remaining area of the **Gameplay UI** is visually de-emphasized by applying a dimmed and blurred (dismissed) effect.

While the **End Game Menu** is active:

- All interactions with the underlying **Gameplay UI** are disabled
- The user can only interact with the elements within the **End Game Menu**

This design ensures that the user's focus is directed toward the game result and available next actions, while still preserving the final board state in the background.

User Input:

- The user selects one of the available options in the **End Game Menu**

Program Output:

The **End Game Menu** includes the following components:

1. Winner Indicator

- Displays the result of the game:
 - White wins
 - Black wins
 - Draw
 - Terminated by user

2. Clock Display

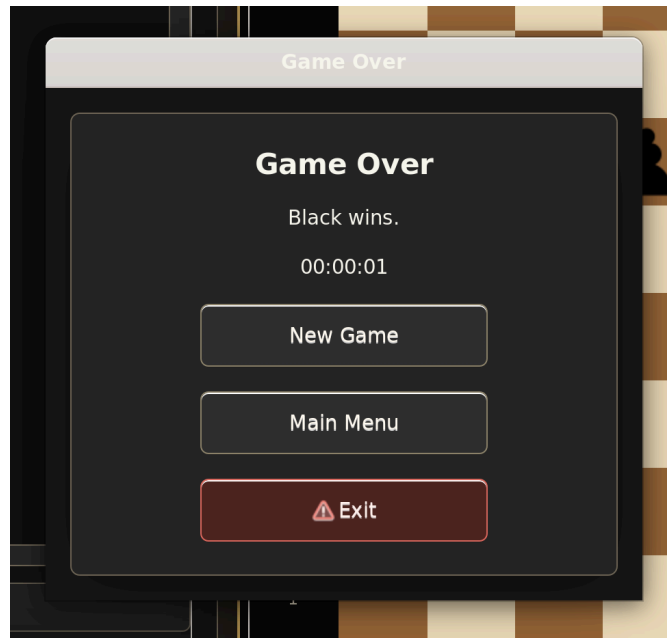
- Displays the total elapsed game time
- The value is retrieved from the [Clock](#) feature described in the Gameplay UI

3. New Game Button

- Redirects the user to the [Game Mode Selection Menu](#)
- Starts a new game setup process

4. Back to Main Menu Button

- Returns the user to the [Main Menu](#)



3.8. Error Handling

This section describes how the program responds to errors that may occur during gameplay.

The Anteater Chess program distinguishes between **non-fatal errors** and **fatal errors**.

- **Non-fatal errors** are errors from which the program can recover immediately without interrupting the game session. These errors are shown through a temporary **Error Message Popup**.
- **Fatal errors** are unexpected internal failures that prevent the current game session from continuing safely. These errors are shown through a **Fatal Error Window**, after which the user is returned to the Main Menu.

This design ensures that minor user mistakes are handled with lightweight feedback, while severe system-level failures are handled safely and explicitly.

3.8.1. Error Message Popup

Description:

The **Error Message Popup** is used to display recoverable errors that occur during normal gameplay, typically caused by invalid user actions or rule violations.

The popup appears briefly on the screen and disappears automatically without interrupting the game flow.

User Input:

- The user performs an invalid or disallowed action during gameplay

Program Output

- A temporary [message](#) is displayed to indicate the error
- The message explains the issue in a user-friendly manner
- The popup disappears automatically after a short duration
- The game continues without interruption
- The user may retry the action

Behavior Rules:

- The popup does not require user confirmation
- The popup does not pause or terminate the game

3.8.2. Fatal Error Window

Description:

The **Fatal Error Window** is used to handle rare and unexpected system-level failures that prevent the current game session from continuing safely.

To maintain a user-friendly experience, the program does not expose technical details of the failure. Instead, a general message is displayed, and the user is guided to exit the current session in a controlled manner.

User Input:

- The user confirms the message displayed in the window

Program Output:

- A **modal window** appears with a general error message, such as:
 - “A fatal error occurred.”
 - “The current game session cannot continue.”
- The window requires user confirmation
- After confirmation:
 - The current game session is terminated safely
 - The user is returned to the [Main Menu](#)

Behavior Rules:

- Fatal error causes are not displayed to the user
- The program avoids abrupt crashes by handling the error through this window

- The window must be acknowledged before exiting the current session
-

3.9. Move Log

Description:

The **Move Log** is a system component responsible for recording all state-changing events during a game into a **human-readable .log text file**.

The log serves as a persistent record of the entire game and reflects the exact sequence of moves that define the current board state. It mirrors the in-memory move history used by the game logic and remains fully consistent with the current game state.

The **Move Log** follows a state-consistent design: only events that modify the game state are recorded. Non-state-changing interactions (such as invalid moves, piece selection, or highlighting) are not included.

The log file is created automatically when a new game begins and is updated incrementally throughout gameplay. All game logic and move history are maintained in memory, and the log file serves only as a persistent representation of this data.

User Input:

- The user does not directly interact with the **Move Log**
- The log is automatically updated by the system in response to gameplay events, including [Valid Moves](#), [Undo operations](#), and [Game Termination](#)

Program Output:

Log File Lifecycle

- A new .log file is created automatically at the start of each game
 - Each game generates a **separate log file**
 - The file is updated incrementally after each valid state-changing event
 - The file remains available after the game ends for review or archival purposes
-

Event Triggers

The **Move Log** is updated only when state-changing events occur:

- Game Start
- A successful **Move Validation** process
- **Undo** Operation (updates the move history and triggers log reconstruction)
- A **Game Termination Status** occurs

Events that do NOT trigger logging include:

- Invalid move attempts
- Piece selection or highlighting
- Incomplete input

Data Flow Model

The **Move Log** follows a layered data flow design:

- **Memory Layer:**
 - All game state updates and move history modifications are performed in memory
- **File Layer:**
 - The log file is updated only after memory changes are completed, ensuring consistency

This separation ensures that gameplay execution remains efficient while maintaining a reliable, persistent record.

Logging Behavior

Game Start:

- Memory Operation:
 - The game state and move history are initialized in memory
- File Operation:

The log begins with a game header, which may include:

 - Game Start
 - Game mode (e.g., Human vs Computer)

Valid Move:

- Memory Operation:
 - The validated move is applied to the in-memory game state and appended to the move history list
- File Operation:
 - Each valid move is appended as a new entry in the log file

- The entry includes structured, human-readable information
- Computer moves are explicitly labeled (e.g., Black (AI))

Undo Operation:

- Memory Operation:
 - The system updates the in-memory move history by reverting the most recent move
- File Operation:
 - The log file is reconstructed based on the updated move list
 - This ensures the log remains fully consistent with the current board state
 - No additional undo record is written

Game Termination:

- Memory Operation:
 - The final game state is preserved and the result is determined
- File Operation:
 - A final entry is appended, indicating the game result

Examples:

 - Result: White wins
 - Result: Draw
 - Result: Game terminated by user
 - The total game time (from the Clock feature) is also recorded

Log Entry Format

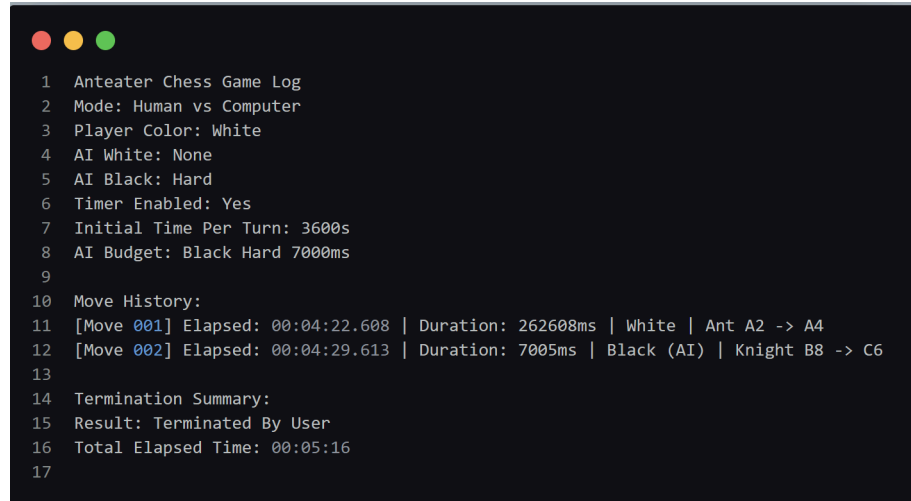
Each move entry is **structured** and **human-readable**. A typical entry includes:

- Move Number
- Timestamp (elapsed game time)
- Player (including AI if applicable)
- Piece Type
- Origin and Destination Squares
- Capture Information (if any)
- Special Move Annotation (if applicable)

Example Entry:

[Move 01] 00:00:05 | White Pawn F2 → F4
[Move 02] 00:00:12 | Black (AI) Pawn E7 → E5
[Move 03] 00:00:20 | White Knight G1 → F3
[Move 04] 00:00:28 | Black (AI) Pawn E5 → F4 | Capture: White Pawn

Example Full Log:



```
1 Anteater Chess Game Log
2 Mode: Human vs Computer
3 Player Color: White
4 AI White: None
5 AI Black: Hard
6 Timer Enabled: Yes
7 Initial Time Per Turn: 3600s
8 AI Budget: Black Hard 7000ms
9
10 Move History:
11 [Move 001] Elapsed: 00:04:22.608 | Duration: 262608ms | White | Ant A2 -> A4
12 [Move 002] Elapsed: 00:04:29.613 | Duration: 7005ms | Black (AI) | Knight B8 -> C6
13
14 Termination Summary:
15 Result: Terminated By User
16 Total Elapsed Time: 00:05:16
17
```

Integration with Gameplay UI

The [Move History Panel](#) in the **Gameplay UI** displays the move history stored in **Memory Layer**.

The **Move Log** file serves as a persistent storage layer, while the UI reads from the in-memory move list to ensure efficient rendering and real-time updates. The log file is not used as a runtime data source and does not affect gameplay performance.

This design separates data storage from presentation:

- The log file provides a complete, human-readable record
- The UI provides an interactive and scrollable view of the same data

File Format

The **Move Log** is stored as a **plain text file with a .log extension**.

The file is fully **human-readable** and can be opened with any standard text editor.

No specialized software is required to interpret the log contents.

4. Back Matter

4.1. Troubleshooting

- **Problem:** The game does not start, or the GUI does not appear
- **Solution:** Ensure your system meets the minimum requirements, ensure the proper toolchains are installed, and the versions are up to date.

4.2. Error Messages

1. Input Errors

Invalid Command Input

Trigger:

The user enters an incomplete, malformed, or unrecognized move command.

Error Message:

“Invalid input.”

Outcome:

No move is executed. The user must enter a new command.

2. Selection Errors

Empty Tile Selection

Trigger:

The user attempts to move from a tile that does not contain a piece.

Error Message:

“No piece here.”

Outcome:

No move is executed. The user must select another tile.

Opponent Piece Selection

Trigger:

The user attempts to move a piece that belongs to the opponent.

Error Message:

“Not your piece.”

Outcome:
No move is executed. The user must select one of their own pieces.

3. Move Validation Errors

Illegal Move

Trigger:
The attempted move does not satisfy the game's movement rules.

Error Message:
"Illegal move."

Outcome:
The move is rejected. The user must try again.

Unresolved Check

Trigger:
The player is currently in check, and the attempted move does not remove that check.

Error Message:
"Resolve check first."

Outcome:
The move is rejected. The user must make a move that resolves the check.

4. Action Availability Errors

Invalid Undo Request

Trigger:
The user requests an undo when undo is not available, such as when there are no moves left to undo or when it is currently the computer's turn.

Error Message:
"Undo unavailable."

Outcome:
No change is made to the board state.

Invalid Hint Request

Trigger:

The user requests a hint when hints are not available, such as during the computer's turn.

Error Message:
"Hint unavailable."

Outcome:
No hint is displayed.

Turn Violation

Trigger:
The user attempts to perform a move or gameplay action during the computer's turn.

Error Message:
"Not your turn."

Outcome:
The action is ignored and no change is made to the board.

5. Timer Events

Turn Timeout

Trigger:
The player does not make a move before the turn timer reaches zero.

Error Message:
"Time's up."

Outcome:
The current turn ends and control passes to the opponent.

4.3. Copyright and contact information

Software Title: Anteater Chess

Course: EECS 22L: Software Engineering Project in C, Spring 2026

Institution: University of California, Irvine

Development Team (Team 22: DeepAnteater):

Yao Li	yaol64@uci.edu
Yurang Li	yurangl@uci.edu
Yide Li	yidel3@uci.edu
Yasith Diunugala	ydiunuga@uci.edu
Max Zhang	maxyz@uci.edu
Benjamin Feng	benjaf3@uci.edu

Contact & Support: For software issues, please contact Team 22

4.4. Index

Glossary	Page
AI Difficulty Levels	11, 17-19
Ant	3-4
Ant Eating	3, 30
Anteater	3-4, 6, 30
Bishop	3-5
Castling	3, 30
Check	3, 31, 37
Checkmate	3-4, 9, 33
Clock	11, 22, 35, 39-40
En Passant	4, 30
End Game Menu	35-36

Error Messages	29, 32, 36-38
Game Mode Selection	14-19
Game Setup Menu	17-20
GamePlay UI	20-24, 34-35
Hint Function	11, 24, 37
Installation	13
King	4, 9, 31, 33
Knight	4-5
Main Menu	14-17, 36
Move History Panel	11, 23, 40
Move Input Widget	22, 24-26
Move Log	38-41
Promotion	4, 30
Special Moves	30
Stalemate	5, 9
System Requirements	13
Turn Timer	17–20, 23, 31–33, 38
Troubleshooting	42
Turn Indicator	7, 21, 31–32
Undo	11, 23, 31, 33, 39
Uninstalling	13
Usage Scenarios	7–10